Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Il problema dell'ordinamento: Ordinamenti naif

Tiziana Calamoneri

SAPIENZA
UNIVERSITÀ DI ROMA

Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

IL PROBLEMA DELL'ORDINAMENTO

PROPRIETÀ

Il problema dell'ordinamento

RICORRENTE · FONDAMENTALE · SOTTOPROBLEMA di tanti PROBLEMI REALI

Il problema dell'**ordinamento** degli elementi di un insieme è (1) un problema molto ricorrente in informatica poiché ha (2) un'importanza fondamentale per le applicazioni: lo si ritrova molto frequentemente come (2) sottoproblema nell'ambito dei problemi reali.

IMPATTO SUL TEMPO di CALCOLO COMPLESSIVO

Si stima che una parte rilevante del tempo di calcolo complessivo consumato nel mondo sia relativa all'esecuzione di algoritmi di ordinamento.

NECESSITÀ SOLUZIONE EFFICIENTE

É NECESSARIO QUINDI SVILUPPARE ALGORITMI CHE SIANO IN GRADO DI RISOLVERE QUESTO PROBLEMA NEL MODO + EFFICIENTE POSSIBILE.

ALGORITMI di ORDINAMENTO

Algoritmi di ordinamento - 1

DEFINIZIONE

Un **algoritmo di ordinamento** è un (1) algoritmo capace di ordinare gli elementi di un insieme (2) sulla base di una certa relazione d'ordine, definita sull'insieme stesso.

INSIEME di DATI

Modello semplificato ~ VETTORE INTERI

Per semplicità di trattazione, supponiamo che gli *n* elementi da ordinare siano **numeri interi** e siano contenuti in un **vettore** i cui indici vanno da *1* a *n*.

Modello Reale ~ RECORDS

Tuttavia, nei problemi reali, i dati da ordinare sono ben più complessi: in generale essi sono strutturati in ***record***, (*) cioè in gruppi di informazioni non sempre omogenee relative allo stesso soggetto, e si vuole ordinarli rispetto ad una di tali informazioni (ad esempio, il Codice Fiscale). (*): VEDI APPUNTI/NOTE

Algoritmi di ordinamento - 2

TIPOLOGIE

Esistono diversi algoritmi di ordinamento.

Dapprima illustreremo gli algoritmi più semplici ma ci renderemo conto che, volendo migliorare l'efficienza, sarà necessario mettere in campo idee più elaborate.

SEMPLICI · Insertion Sort ~ Selection Sort ~ Bubblesort ~

Gli algoritmi semplici che illustreremo sono:

Insertion sort Selection sort Bubblesort

EVOLUTI · Mergesort ~ Quicksort ~ Heapsort ~

Gli algoritmi più evoluti che vedremo sono:

Mergesort Quicksort Heapsort

INSERTION SORT

PROCEDIMENTO

Insertion sort - 1

Steps:

1. Gli elementi da ordinare sono inizialmente contenuti nel vettore. Per ogni $i=2, \ldots, n$:
2. Si estrae l'elemento della posizione i, così da liberare la sua posizione corrente
3. Si spostano di una posizione verso destra tutti gli elementi alla sua sinistra (già ordinati) che sono maggiori di esso
4. Si inserisce l'elemento nella posizione che si è liberata.

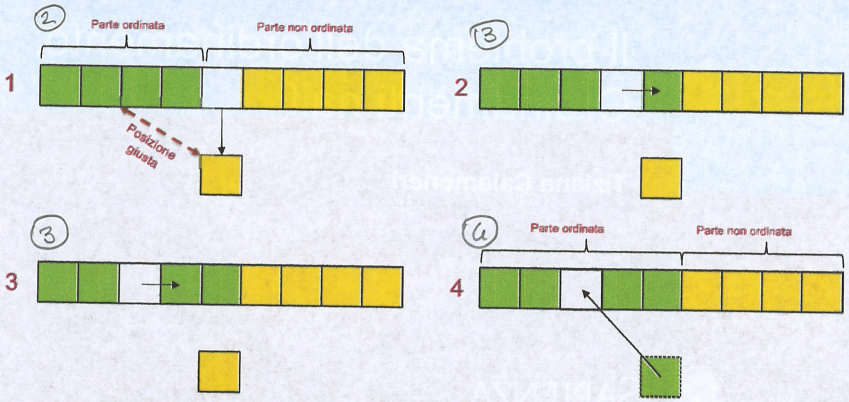
In Sintesi:

- INVARIANTE: ad ogni passo i, gli elementi con indice $\leq i$ (a sinistra) sono già ordinati, mentre quelli con indice $> i$ (a destra) sono ancora da processare.

Graficamente:

Insertion sort - 2

L'algoritmo procede per iterazioni, aumentando ad ogni iterazione la dimensione della porzione di sinistra del vettore, che è ordinata.



ESEMPIO

Insertion sort - 3

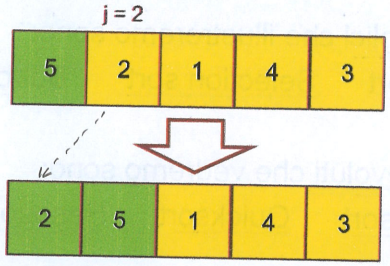
Prima di esaminare il codice dell'algoritmo vediamo un esempio.

Caso Base:

- Un vettore di un solo elemento è già ordinato, quindi si estrae per primo il secondo elemento. L'indice usato nel codice per l'elemento da sistemare è j

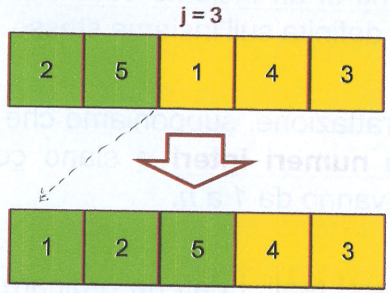
Steps:

1



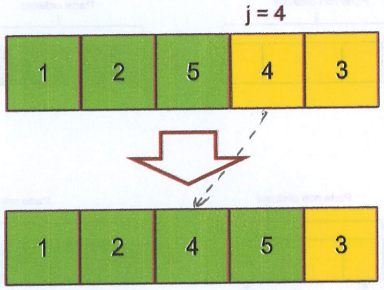
Insertion sort - 4

2 Nella successiva iterazione $j = 3$



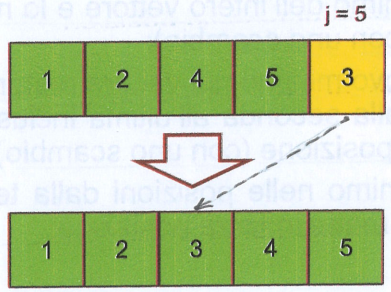
Insertion sort - 5

③ Nella successiva iterazione $j = 4$



Insertion sort - 6

④ Nell'ultima iterazione, per $j = 5$:



Dopodiché il vettore risulta ordinato.

Note: (A) SI PARTE DAL 2° ELEMENTO PERCHÉ IL PRIMO È GIÀ ORDINATO DA SOLO.
(B) SALVO L'ELEMENTO DA ORDINARE A[j] IN UNA VARIABILE AUSILIARIA x.
(C) INIZIALIZZO L'INDICE DA USARE PER SCORRERE GLI ELEMENTI ALLA SX DI A[j]
(D) FINCHÉ L'EL A[i] È > x, LO SPOSTO VERSO DX
(E) QUANDO IL LOOP TERMINA, POSSIAMO INSERIRE x NELLA SUA POSIZIONE CORRETTA.

Insertion sort - 7

Lo pseudocodice dell'algoritmo è il seguente.

PSEUDOCODICE

Python

```
Funzione Insertion_Sort(A[1..n])
  for j = 2 (A) to n do                    (n-1) Θ(1) + Θ(1)
    x ← A[j] (B)                           Θ(1)
    i ← j - 1 (C)                          Θ(1)
    while ((i>0) and (A[i])>x) (D)         t_j Θ(1) + Θ(1)
      A[i+1] ← A[i] (D)                    Θ(1)
      i ← i - 1 (D)                        Θ(1)
    A[i+1] ← x (E)                         Θ(1)
```

PER ORA USIAMO UNA VARIABILE GENERICA t_j IL CUI VALORE VARIA IN BASE AL j-ESIMO CICLO NEL QUALE CI TROVIAMO.

Costo computazionale:

COSTO COMPUTAZIONALE

Formula: $T(n) = \sum_{j=2}^{n}\left(\Theta(1) + t_j\Theta(1) + \Theta(1)\right) + \Theta(1)$

Insertion sort - 8

Caso migliore e peggiore per t_j

- $T(n) = \sum_{j=2}^{n}\left(\Theta(1) + t_j\Theta(1) + \Theta(1)\right) + \Theta(1)$

Il numero di iterazioni del while interno, t_j, può andare:

- da un minimo di *1* per ogni *j* (se ogni x > A[j-1]) ? ORDINATO
- a un massimo di *j* per ogni *j* (se ogni x < A[1]) ? ORDINATO AL CONTRARIO

Quindi caso migliore e caso peggiore differiscono:

Risultati.

$\Omega(n)$ Caso migliore: $T(n) = (n-1)\Theta(1) = \Theta(n)$

$O(n^2)$ Caso peggiore: $T(n) = \sum_{j=2}^{n}\left(\Theta(1) + \Theta(j)\right) =$

$= \Theta(n) + \Theta\left(\frac{n(n+1)}{2} - 1\right) = \Theta(n^2)$

SELECTION SORT

Selection sort - 1

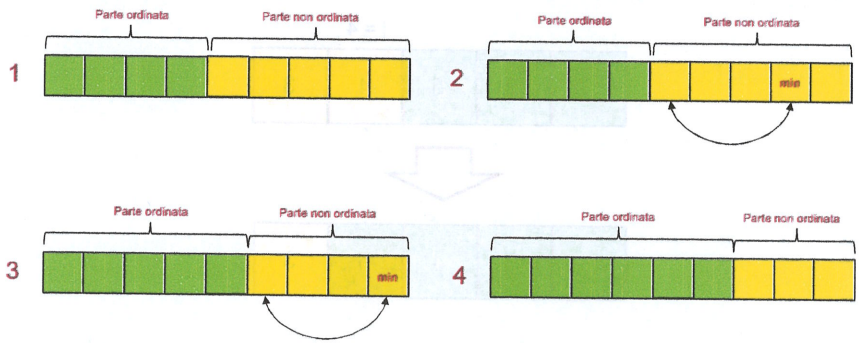
PROCEDIMENTO:

Steps:

1. Cerca il minimo dell'intero vettore e lo mette in prima posizione (con uno scambio);
2. Cerca il nuovo minimo nel vettore restante, cioè nelle posizioni dalla seconda all'ultima incluse, e lo mette in seconda posizione (con uno scambio);
3. Cerca il minimo nelle posizioni dalla terza all'ultima incluse e lo mette in terza posizione;
4. e così via...

Selection sort - 2

Graficamente:



ecc.

② TROVA IL MINIMO NELLA PARTE NON ORDINATA E LO SPOSTA IN PRIMA POSIZIONE.

③ TROVA IL MINIMO NELLA PARTE NON ORDINATA E LO SPOSTA IN PRIMA POSIZIONE.

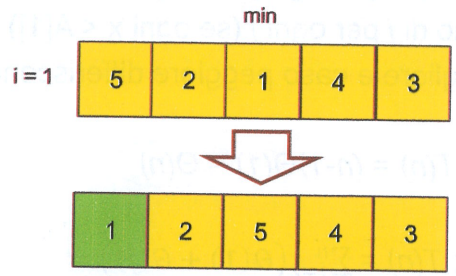
ESEMPIO:

Selection sort - 3

Steps:

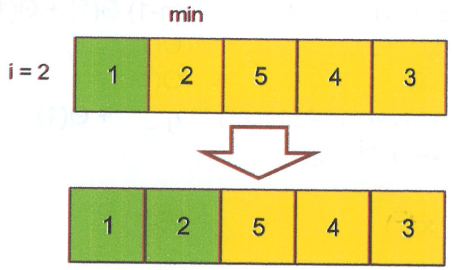
Prima di esaminare il codice dell'algoritmo vediamo un esempio.

1. L'indice usato nel codice per l'elemento da sistemare è *i*



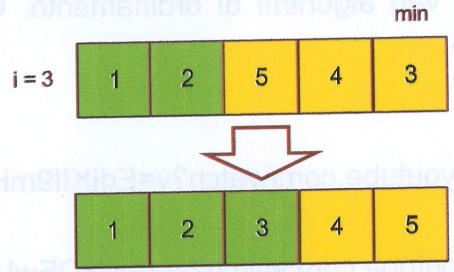
Selection sort - 4

2. Per ***i* = 2** il minimo, in questo esempio, viene rimesso nella sua posizione:



Selection sort - 3

③ Successiva iterazione, per ***i = 3***:



E così via. In questo esempio le ultime due iterazioni rimettono ciascuno dei due elementi al proprio posto

PSEUDOCODICE

Selection sort - 5

Lo pseudocodice dell'algoritmo è il seguente.

```
Funzione Selection_Sort(A[1..n])
  for i = 1 to n - 1 do                    (n - 1) Θ(1) + Θ(1)
    m ← i                                  Θ(1)
    for j = i + 1 to n do                  (n - i) Θ(1) + Θ(1)
      if (A[j] < A[m])                     Θ(1)
        m ← j                              Θ(1)
    Scambia A[m] e A[i]                    Θ(1)
```

COSTO COMPUTAZIONALE

Formula.

$\mathcal{O}(n^2)$.

Costo computazionale:

$$T(n)=\sum_{i=1}^{n-1}\left(\Theta(1)+(n-i)\Theta(1)+\Theta(1)\right)+\Theta(1)=\sum_{i=1}^{n-1}(i\Theta(1)+\Theta(1))=\Theta(n^2)$$

In questo algoritmo non c'è differenza fra i costi di caso migliore e peggiore.

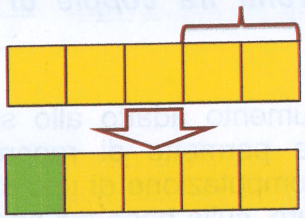
BUBBLE SORT

Origine Nome: L'algoritmo funziona in modo che vede il movimento degli el. simile a quello di bolle d'aria.

PRELIMINARE

Bubble sort - 1

In Sintesi. Funziona per fasi. Ogni fase ispeziona una dopo l'altra, da destra a sinistra, ogni coppia di elementi adiacenti e, se l'ordine dei due elementi non è quello giusto, essi vengono scambiati. (*)

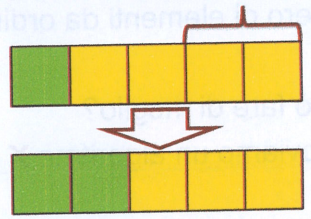


(*): CIÓ CONSENTE, APPUNTO, DI SPOSTARE DA DX A SX IL VALORE MAX... PROPRIO COME UNA BOLLA.

Steps. ① La prima «passata» scende fino alla prima posizione e sistema il minimo nella posizione corretta.

Bubble sort - 2

② La seconda passata si ferma alla seconda posizione e sistema il secondo minimo.



③ E così via per le passate (iterazioni) successive.

Bubble sort - 3

PSEUDOCODICE • Lo pseudocodice dell'algoritmo è il seguente.

```
Funzione Bubble_Sort(A[1..n])
  for i = 1 to n do                    n Θ(1) + Θ(1)
    for j = n downto i + 1 do          (n – i) Θ(1) + Θ(1)
      if (A[j] < A[j - 1])             Θ(1)
        Scambia A[j] e A[j - 1]        Θ(1)
```

COSTO COMPUTAZIONALE • Costo computazionale:

FORMULA • $T(n)=\sum_{i=1}^{n}(\Theta(1)+(n-i)\Theta(1)+\Theta(1))+\Theta(1) = \Theta(n^2)$ (ANCHE $\Omega(n)$ È POSSIBILE!)

$\Theta(n^2)$ • Anche in questo algoritmo non c'è differenza fra i costi di caso migliore e peggiore.

$\Omega(n)$? • ATTENZIONE!! IL BUBBLESORT PUÒ ESSERE $\Omega(n)$ SE SI USA UN PARAMETRO DI CONTROLLO CHE FERMA IL LOOP QUANDO L'ARRAY È ORDINATO!!

ANIMAZIONI ALGORITMI

Animazioni degli algoritmi di ordinamento

In rete sono disponibili diverse pagine che offrono l'animazione di vari algoritmi di ordinamento. Queste sono particolari.

INSERTION SORT • Insertion sort:
https://www.youtube.com/watch?v=EdIKIf9mHk0

SELECTION SORT • Selection sort:
https://www.youtube.com/watch?v=0-W8OEwLebQ

BUBBLE SORT • Bubble sort:
https://www.youtube.com/watch?v=semGJAJ7i74

COMPLESSITÀ ORDINAMENTO

PROBLEMA •

La complessità dell'ordinamento - 1

ALGORITMI INEFFICIENTI • I tre algoritmi di ordinamento appena visti hanno tutti un costo computazionale asintotico che cresce come il quadrato del numero di elementi da ordinare. ($\Theta(n^2)$!!)

IDEE di SOLUZIONE •

SI PUÒ FARE MEGLIO? • Domanda 1: si può fare di meglio?
Risposta: si, se troviamo un algoritmo **X** con costo minore

QUANTO MEGLIO? • Domanda 2: Se si, quanto meglio si può fare?
Risposta: Non sapremmo, chi ci assicura che non si possa fare ancora meglio del nuovo algoritmo **X**?

SOLUZIONE •

La complessità dell'ordinamento - 2

LIMITE INF. COSTO COMPUTAZ. • Come si fa a stabilire un limite di costo computazionale ***al di sotto del quale nessun algoritmo di ordinamento basato su confronti fra coppie di elementi possa andare?***

ALBERO DI DECISIONE •

RAPPRESENTAZIONE STRADE DI SVILUPPO ALGORITMO • Esiste uno strumento adatto allo scopo, l'***albero di decisione***, che permette di rappresentare tutte le strade che la computazione di uno specifico algoritmo può intraprendere, sulla base dei possibili esiti dei test previsti dall'algoritmo stesso.

ALGORITMI DI ORDINAMENTO • Nel caso degli algoritmi di ordinamento basati su confronti, ogni test effettuato ha due soli possibili esiti (ad es.: minore o uguale, oppure no). È BINARIO

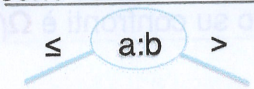
ALBERO di DECISIONE

La complessità dell'ordinamento - 3

Proprietà · Validità · L'albero di decisione relativo a un qualunque algoritmo di ordinamento basato su confronti ha queste proprietà:

Albero Binario ·· è un albero binario che rappresenta tutti i possibili confronti che vengono effettuati dall'algoritmo; ogni nodo interno (ossia un nodo che non è una foglia) ha esattamente due figli;

Nodi ·· ogni nodo interno rappresenta un singolo confronto, ed i due figli del nodo sono relativi ai due possibili esiti di tale confronto;

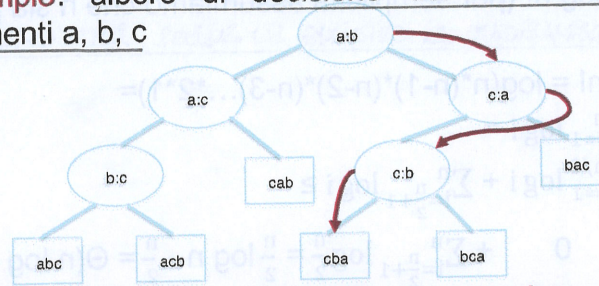


Foglie ·· ogni foglia rappresenta una possibile soluzione del problema, la quale è una specifica permutazione (*) della sequenza in ingresso. (*) COMBINAZIONE ORDINATA

ESEMPIO · Insertion Sort ·

La complessità dell'ordinamento - 4

Esempio: albero di decisione dell'insertion sort su 3 elementi a, b, c



Cammino · Eseguire l'algoritmo corrisponde a scendere dalla radice dell'albero alla foglia che contiene la permutazione che costituisce la soluzione; la discesa è governata dagli esiti dei confronti che vengono via via effettuati.

Lunghezza Cammino · La lunghezza (= numero degli archi) di tale cammino rappresenta il numero di confronti necessari per trovare la soluzione.

CASO PEGGIORE

La complessità dell'ordinamento - 5

Altezza Albero · La lunghezza del ***percorso più lungo*** dalla radice ad una foglia (***altezza*** dell'albero binario) rappresenta il numero di confronti che l'algoritmo deve effettuare nel caso peggiore.

Limitazione Inferiore · Determinare una limitazione inferiore all'altezza dell'albero di decisione relativo a ***qualunque*** algoritmo di ordinamento basato su confronti equivale a trovare una limitazione inferiore al tempo di esecuzione nel caso peggiore di qualunque algoritmo di ordinamento basato su confronti.

RICERCA LIMITE INFERIORE

La complessità dell'ordinamento - 6

Cerchiamo di determinare tale limitazione:

Osservazione n! foglie · Osservazione 1: dato che la sequenza di ingresso può avere una qualunque delle sue permutazioni come soluzione, l'albero di decisione ***deve*** contenere nelle foglie tutte le permutazioni della sequenza in ingresso, che sono $n!$ per un problema di dimensione n.

max 2^h foglie ·· Osservazione 2: un albero binario di altezza h non può contenere più di 2^h foglie.

Condizione $2^h \geq n!$ · Segue che l'altezza h dell'albero di decisione di ***qualunque*** algoritmo di ordinamento basato su confronti deve essere tale per cui:

$2^h \geq n!$ ossia $h \geq \log n!$

La complessità dell'ordinamento - 7

Risoluzione.

Valutiamo log n! (per semplicità assumiamo che n sia pari).

* VEDI APPROSSIMAZIONE DI STIRLING SU SILLES UNITELMA *

$$\log n! = \log(n*(n-1)*(n-2)*(n-3)\ldots*2*1)=$$
$$= \sum_{i=1}^{n} \log i =$$
$$= \sum_{i=1}^{n/2} \log i + \sum_{i=\frac{n}{2}+1}^{n} \log i \geq$$
$$\geq \quad 0 \quad + \sum_{i=\frac{n}{2}+1}^{n} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} = \Theta(n \log n)$$

Limite Inferiore.

Siccome abbiamo visto che $h \geq \log n!$
e che $\log n! = \Theta(n \log n)$
ciò significa che

$$h = \Omega(n \log n)$$

La complessità dell'ordinamento - 8

TEOREMA LIMITE INFERIORE.

Quanto detto si riassume nel seguente:

Teorema.
Il costo computazionale di qualunque algoritmo di ordinamento basato su confronti è $\Omega(n \log n)$.

CIÓ SIGNIFICA CHE, SE TROVIAMO UN ALGORITMO DI ORDINAMENTO, BASATO SU CONFRONTI, CHE ABBIA UN COSTO COMPUTAZIONALE $\Theta(n \log n)$... NON POTREMO MAI TROVARNE UNO MIGLIORE!

ESERCIZIO SVOLTO.

Esercizio svolto - 1

Testo.

Nell'algoritmo di Insertion sort è possibile ricercare la posizione in cui inserire l'elemento *i*-esimo tramite la ricerca binaria. Come cambia il calcolo del costo computazionale dell'algoritmo?

Soluzione.

Soluzione. Per ogni j = 2 to n, l'algoritmo di Insertion Sort in versione standard:

- scorre la parte ordinata di vettore da destra a sinistra
- confronta ciascun elemento con x
- trova la posizione corretta di x
- sposta tutti gli elementi che sono a destra per fare posto ad x

Esercizio svolto - 2

Consideriamo lo pseudocodice modificato dell'Insertion Sort

```
Funzione Insertion_Sort(A[1..n])
  for j = 2 to n do                          (n-1) Θ(1) + Θ(1)
    x ← A[j]                                 Θ(1)
    k ← RicercaBin(A,x)  (*)                 O(log j)
        /* x dovrebbe stare in pos.k
    i ← j - 1                                Θ(1)
    while ((i>0)and(i>=k))                   t_j Θ(1)+ Θ(1)
      A[i+1] ← A[i]                          Θ(1)
      i ← i - 1                              Θ(1)
  A[k] ← x
```

(*): SI USA L'ALGORITMO DI RICERCA BINARIA PER TROVARE A QUALE INDICE SPOSTARE L'ELEMENTO

Esercizio svolto - 3

- Il contributo della ricerca binaria non si sostituisce a t_j, ma si aggiunge ad esso.
- Anche se troviamo la posizione di x più velocemente, dobbiamo comunque spostare gli elementi a destra…
- pertanto il costo computazionale non migliora!

Corso di laurea in Informatica
Insegnamento di Introduzione agli Algoritmi
Lezioni a distanza

Esercizi per casa

SAPIENZA
UNIVERSITÀ DI ROMA

Esercizi - 1

1. Si consideri l'algortimo Bubble sort e si sostituisca il ciclo interno for (j=n downto i+1) con for (j=n downto 1). Funziona ancora correttamente? Il costo computazionale dell'algoritmo cambia?
2. Un algoritmo di ordinamento si dice stabile se, in caso in cui ci siano più occorrenze dello stesso valore nel vettore, esso le lascia nell'ordine in cui si trovano originariamente. Qualcuno tra i tre algoritmi visti in questa lezione è stabile? *VEDI APPUNTI x MIGLIORE DEF. DI STABILITÀ*
3. Qual è il costo computazionale dei tre algoritmi visti in questa lezione quando il vettore di input ha gli elementi che sono tutti uguali? e quando è già ordinato?

Esercizi - 2

4. Scrivere una funzione che, dato un vettore di *n* elementi ed un indice *j*, trovi il minimo del sottovettore *A[j..n]*. Riscrivere lo pseudocodice del Selection sort sfruttando questa funzione.
5. Dato un vettore di *n* elementi, si progetti un algoritmo che verifichi se ci sono occorrenze ripetute di uno stesso valore (e, ad esempio, restituisca 1 se ve ne sono e 0 altrimenti).

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Il problema dell'ordinamento: Il Merge Sort

Tiziana Calamoneri

SAPIENZA
UNIVERSITÀ DI ROMA

Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

OTTIMIZZAZIONE
ORDINAMENTO

Nella precedente lezione...

...abbiamo dimostrato il seguente:

LIMITE INFERIORE
- **Teorema**. Il costo computazionale di qualunque algoritmo di ordinamento basato su confronti è $\Omega(n \log n)$.

ALGORITMI OTTIMI
- Riusciamo a progettare degli algoritmi che richiedono costo computazionale uguale proprio a $\Theta(n \log n)$ e sono, quindi, **ottimi?**
  - MERGESORT
  - QUICKSORT
  - HEAPSORT

MERGE SORT

Merge Sort - 1

PROPRIETÀ
- Ricorsione
- Divide Et Impera

L'algoritmo ***merge sort*** (***ordinamento per fusione***) è un algoritmo ricorsivo che adotta una tecnica algoritmica detta ***divide et impera***. Essa può essere descritta come segue:

DIVIDE ET IMPERA
- Divide: il problema complessivo si suddivide in sottoproblemi di dimensione inferiore (***divide***);
- Impera: i sottoproblemi si risolvono ricorsivamente (***impera***);
- Combina: le soluzioni dei sottoproblemi si compongono per ottenere la soluzione al problema complessivo (***combina***).

Merge Sort - 2

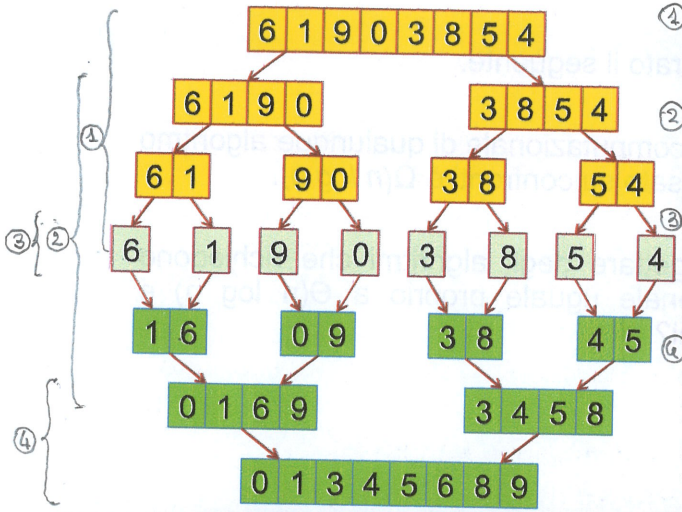
PROCEDIMENTO

L'approccio dell'algoritmo Merge Sort è il seguente:

STEPS
1. DIVIDE: ***divide***: la sequenza di n elementi viene divisa in due sottosequenze di $n/2$ elementi ciascuna;
2. IMPERA: ***impera***: le due sottosequenze di $n/2$ elementi vengono ordinate ricorsivamente;
3. PASSO BASE: ***passo base:*** la ricorsione termina quando la sottosequenza è costituita di un solo elemento, per cui è già ordinata;
4. COMBINA: ***combina***: le due sottosequenze – ormai ordinate – di $n/2$ elementi ciascuna vengono "fuse" in un'unica sequenza ordinata di n elementi.

Graficamente.

Merge Sort - 3



① **divide**: la sequenza di *n* elementi viene divisa in due sotto-sequenze di *n/2* elementi ciascuna;

② **impera**: le due sotto-sequenze di *n/2* elementi vengono ordinate ricorsivamente;

③ **passo base:** la ricorsione termina quando la sotto-sequenza è costituita di un solo elemento, per cui è già ordinata;

④ **combina**: le due sotto-sequenze – ormai ordinate – di *n/2* elementi ciascuna vengono "fuse" in un'unica sequenza ordinata di *n* elementi.

Pseudocodice.

Merge Sort - 4

```
Funzione Merge_sort (A: vettore; ind_primo,
                     ind_ultimo: intero)                    T(n)
  if (ind_primo < ind_ultimo)                               Θ(1)
    ind_medio ← (ind_primo+ind_ultimo) DIV 2                Θ(1)
    Merge_sort (A, ind_primo, ind_medio)                    T(n/2)
    Merge_sort (A, ind_medio + 1, ind_ultimo)               T(n/2)
    Fondi (A, ind_primo, ind_medio, ind_ultimo)             S(n)
  return
```

Costo computazionale.

$T(n)=\Theta(1) + 2T(n/2) + S(n)$ dove $S(n)$ = costo di Fondi()

$T(1)=\Theta(1)$

(p.1)

NOTA: EQUAZIONE DI RICORRENZA RISOLVIBILE TRAMITE IL METODO PRINCIPALE (TEOREMA MASTER)

Funzione Fondi (Merge)

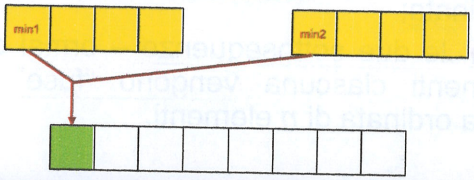
Funzionamento

Merge Sort - 5

Funzionamento della funzione `Fondi()`:

1. Ordina sottosequenze
2. Confronto minimi sottosequenze

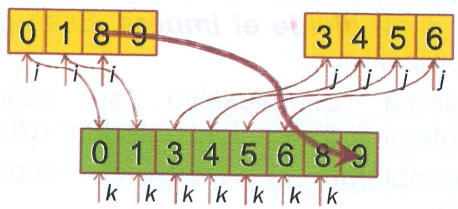
- sfrutta il fatto che le due sottosequenze sono ordinate;
- il minimo della sequenza complessiva non può che essere ***il più piccolo fra i minimi delle due sottosequenze*** (se essi sono uguali, scegliere l'uno o l'altro non fa differenza);
- dopo aver eliminato da una delle due sottosequenze tale minimo, la proprietà rimane: il prossimo minimo non può che essere il più piccolo fra i minimi delle due parti rimanenti delle due sottosequenze.



Esempio.

Merge Sort - 6

Esempio di funzionamento della funzione `Fondi()`:



Dopo aver ricopiato anche l'ultimo elemento il vettore complessivo risulta ordinato.

PSEUDOCODICE.

Merge Sort - 7

```
Funzione Fondi (A: vettore; ind_primo,ind_medio,ind_ultimo: int)
   i ← ind_primo;  j ← ind_medio + 1;  k ← 1;
   while ((i≤ind_medio) and (j≤ind_ultimo))
      if (A[i]<A[j])
         B[k] ← A[i]; i ← i + 1
      else
         B[k] ← A[j]; j ← j + 1
      k ← k + 1
   while (i≤ind_medio) //il primo sottovettore non è terminato
      B[k] ← A[i]; i ← i + 1; k ← k + 1
   while (j≤ind_ultimo) //il secondo sottovettore non è terminato
      B[k] ← A[j]; j ← j + 1; k ← k + 1
   ricopia B[1..k-1] su A[ind_primo..ind_ultimo]
   return
```

← È PIÙ EFFICIENTE SCRIVERLO 1 VOLTA SOLA, FUORI DA IF-THEN-ELSE

Dove t+s+v=n ⇒ T(n)=Θ(n)
...oppure, si può anche vedere come segue...

Ⓐ Θ(n)
Ⓑ Θ(n)
Ⓒ Θ(n)

COSTO COMPUTAZIONALE

Merge Sort - 8

T(n)
Θ(1)
t Θ(1)+Θ(1) Ⓐ
Θ(1)
Θ(1)
Θ(1)
Θ(1)
Θ(1)
s Θ(1)+Θ(1) Ⓑ
Θ(1)
v Θ(1)+Θ(1) Ⓑ
Θ(1)
Θ(n) Ⓒ

Valutiamo il costo computazionale della funzione `Fondi()`:

- inizializzazione delle variabili: $\Theta(1)$;
- primo ciclo `while`
  - ogni iterazione ha costo $\Theta(1)$ e incrementa di 1 l'indice *i* oppure l'indice *j*. Quindi il costo del `while` varia da un minimo di $n/2$ a un massimo di n, ossia è $\Theta(n)$.
- secondo e terzo `while` (mai eseguiti entrambi):
  - si ricopia nel vettore B l'eventuale "coda" di una delle due sottosequenze: $O(n)$;
- copia del vettore B nell'opportuna porzione del vettore A: $\Theta(n)$.

Dunque il costo S(n) della funzione Fondi() è:

$$S(n) = \Theta(1) + \Theta(n) + O(n) + \Theta(n) = \Theta(n)$$

COSTO COMPUTAZIONALE (p.2)

Merge Sort - 9

Quindi il costo computazionale del Merge Sort:

PSEUDOCODICE.

```
Funzione Merge_sort (A: vettore; ind_primo, ind_ultimo)
   if (ind_primo < ind_ultimo)
      ind_medio ← (ind_primo +ind_ultimo) DIV 2
      Merge_sort (A, ind_primo, ind_medio)
      Merge_sort (A, ind_medio + 1, ind_ultimo)
      Fondi (A, ind_primo, ind_medio, ind_ultimo)
   return
```

COSTO.

Θ(nlogn) è

$$T(n)=2T(n/2)+\Theta(n)$$
$$T(1)=\Theta(1)$$

$\Rightarrow$ $T(n)=\Theta(n \log n)$

IMPORTANTE!!: Vedi codice Python MergeSort.py

SVANTAGGI OPERAZIONE NON IN LOCO

Merge Sort - 10

OSSERVAZIONE

T(n)
Θ(1)
Θ(1)
T(n/2)
T(n/2)
Θ(n)
Θ(1)

Non Consentita

- L'operazione di fusione non si può fare "in loco", cioè aggiornando direttamente il vettore A, senza incorrere in un aggravio del costo.

Lievitazione Costo

- Infatti, in A bisognerebbe fare spazio via via al minimo successivo, ma questo costringerebbe a spostare di una posizione tutta la sottosequenza rimanente per ogni nuovo minimo, il che costerebbe $\Theta(n)$ operazioni elementari per ciascun elemento da inserire, facendo lievitare quindi il costo computazionale della fusione da $\Theta(n)$ a $\Theta(n^2)$.

Θ(n²).

- Ciò a sua volta risulterebbe nell'equazione di ricorrenza:

$$T(n) = 2\,T(n/2) + \Theta(n^2)$$

la cui soluzione, come sappiamo, è:

$$T(n) = \Theta(n^2)$$

Esercizio svolto - 1

Esercizio. Nonostante Merge Sort funzioni in tempo $\Theta(n \log n)$ mentre Insertion Sort in $O(n^2)$, i fattori costanti sono tali che l'Insertion Sort è più veloce del Merge Sort per valori piccoli di n. Quindi, ha senso usare l'Insertion Sort dentro il Merge Sort quando i sottoproblemi diventano sufficientemente piccoli.

- Si consideri una modifica del Merge Sort in cui il caso base si applica ad una porzione del vettore di lunghezza k, che viene ordinata usando Insertion Sort.
- Le porzioni vengono combinate usando il meccanismo standard di fusione.
- Si determini il valore di k come funzione di n per cui l'algoritmo modificato ha lo stesso tempo di esecuzione asintotico del Merge Sort.

Esercizio svolto - 2

Soluzione. Il codice che realizza tale versione è il seguente.

```
Funzione Merge_Insertion (V, k, primo, ultimo)
   dim ← ultimo - primo + 1
   if dim > k
      medio ← (primo + ultimo) DIV 2
      Merge_Insertion (V, k, primo, medio)
      Merge_Insertion (V, k, medio + 1, ultimo)
      Fondi(primo, medio, ultimo)
   else InsertionSort(V, primo, ultimo)
```

La chiamata iniziale sarà:

```
Merge_Insertion (V,k,1,n)
```


Esercizio svolto - 3

L'equazione di ricorrenza che lo caratterizza è la seguente:

$T(n)=2T(n/2)+\Theta(n)$

$T(k)=\Theta(k^2)$

Risolviamola col metodo iterativo:

$T(n) = 2T(n/2) + \Theta(n) =$

$= 2[2T(n/2^2) + \Theta(n/2^1)] + \Theta(n/2^0) =$

$= 2[2[2T(n/2^3) + \Theta(n/2^2)] + \Theta(n/2^1)] + \Theta(n/2^0) =$

$=2^3T(n/2^3) + 2^2\,\Theta(n/2^2) + 2^1\,\Theta(n/2^1) + 2^0\,\Theta(n/2^0)$

$\dots$

$=2^h\,T(n/2^h)+\sum_{i=0}^{h-1} 2^i\Theta\left(\frac{n}{2^i}\right)$

Esercizio svolto - 4

Ci fermiamo incontrando il caso base, il che succede quando $n/2^h = k$, ossia $2^h=n/k \Rightarrow h = \log n/k$

Sostituendo tale valore nell'espressione precedente otteniamo:

$$T(n) = 2^{\log n/k}\,T(n/2^{\log n/k}) + \sum_{i=0}^{\log n/k-1} 2^i\Theta\left(\frac{n}{2^i}\right) =$$

$$= \frac{n}{k}\Theta(k^2) + \Theta\left(n \log \frac{n}{k}\right) =$$

$$= \Theta(nk) + \Theta\left(n \log \frac{n}{k}\right) =$$

$$= \Theta(nk) + \Theta(n \log n - n \log k)$$

Se $k = O(\log n)$ otteniamo:

$T(n) = O(n \log n) + \Theta(n \log n - n \log\log n) = \Theta(n \log n)$

Corso di laurea in Informatica
Insegnamento di Introduzione agli Algoritmi
Lezioni a distanza

Esercizi per casa

SAPIENZA
UNIVERSITÀ DI ROMA

Esercizi

1. Scrivere la versione iterativa dell'algoritmo di Merge sort.
2. Scrivere la versione ricorsiva dell'algoritmo di Fusione.
3. Si supponga(*) di scrivere una variante del Merge sort, chiamata 4-Merge sort che, invece di suddividere il vettore da ordinare in 2 parti (e ordinarle separatamente), lo suddivide in 4 parti, le ordina ognuna riapplicando 4-Merge sort, e le riunifica usando un'opportuna variante 4-Fondi di Fondi (che fa la fusione su 4 sottovettori invece che su 2).
4. Come cambia, se cambia, il costo computazionale di 4-Merge sort rispetto a quello di Merge sort?
5. Come cambia, se cambia, il costo computazionale di un'ulteriore variante k-Merge sort che spezza il vettore in k sottovettori?

(*) NON È RICHIESTO SCRIVERE IL CODICE PER INTERO. AL PIÙ GIUSTO UNO PSEUDOCODICE.
LA VERA DOMANDA RIGUARDA RIFLETTERE SU COME POTREBBE VARIARE IL COSTO COMPUTAZIONALE RISPETTO AL CASO BASE.
- VEDI ES. 4 e 5

▸ VEDI Risoluzioni Esercizi in CODICI PYTHON e su APPUNTI ◂

In Conclusione...

Il MergeSort presenta i seguenti Vantaggi/Svantaggi...

- VANTAGGI
  - ~ COSTO COMPUTAZIONALE OTTIMALE: $\Theta(n \log n)$
- SVANTAGGI
  - ~ OPERA NON IN LOCO!

Per avere un algoritmo avente $\Theta(n \log n)$ e che operi IN LOCO... abbiamo bisogno di guardare al
- QUICK SORT
- HEAPSORT

Ricorsione Vs Iterazione

IL Merge Sort è un ALGORITMO TIPICAMENTE RICORSIVO ma esso può anche essere riscritto in Versione ITERATIVA
- VEDI prossime Slides + APPUNTI

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Il problema dell'ordinamento: Il Quicksort

Tiziana Calamoneri

SAPIENZA
UNIVERSITÀ DI ROMA

Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

SOLUZIONE ES. 1

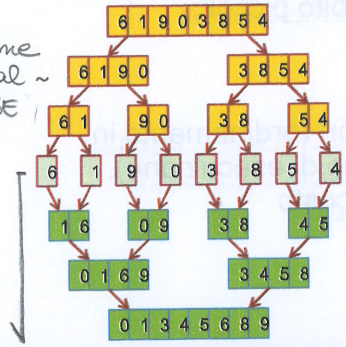
Esercizio Risolto - 1

Trasformazione RICORSIONE ITERAZIONE

Es. Scrivere la versione iterativa del Merge-Sort.

Sol. Sappiamo che è sempre possibile trasformare qualunque algoritmo ricorsivo in un algoritmo iterativo, anche se a volte è meno naturale.

L'iterazione parte dal ~ CASO BASE

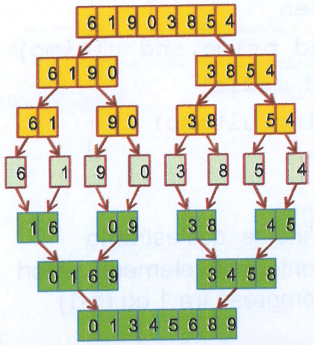


Idea. Possiamo continuare ad usare la funzione Fondi() e basta simulare la parte inferiore nel grafico delle computazioni, visto che la parte superiore mostra semplicemente l'apertura delle chiamate ricorsive.

PSEUDOCODICE

Esercizio Risolto - 2

```
Funzione mergeSortIter(A: vettore, n: int)
  for (lung=1; lung<=n/2; lung*=2)
    for (i=1; i<=n-2*lung+1; i+=2*lung)
      Fondi(A, i, i+lung, i+2*lung-1);
```



Quando `lung`=1 vengono fuse le singole celle
Quando `lung`=2 i vettori da 2 elementi
Quando `lung`=4 …

COSTO COMPUTAZIONALE

Esercizio Risolto - 3

Costo computazionale:

```
Funzione mergeSortIter(A: vettore, n: int)
  for (lung=1; lung<=n/2; lung*=2)        sommatoria?
    for (i=1; i<=n-2*lung+1; i+=2*lung)   n/(2lung) v.
      Fondi(A, i, i+lung, i+2*lung-1);    Θ(lung)
```

Non ci serve usare la sommatoria perché la parte interna del ciclo non dipende da lung:
$T(n)=\log n\ (n/(2\ \texttt{lung})\ \Theta(\texttt{lung})) = \log n\ \Theta(n) = \Theta(n \log n)$

QUICKSORT

PROPRIETÀ •

Quicksort - 1

Caso Peggiore. $O(n^2)$

Caso Migliore. $\Omega(n \log n)$

L'algoritmo ***quicksort*** (***ordinamento veloce***) ha costo $O(n^2)$ nel caso peggiore ma nella pratica è spesso la soluzione migliore per grandi valori di n perché:

- il suo tempo di esecuzione atteso è $\Theta(n \log n)$;
- i fattori costanti nascosti sono molto piccoli;
- permette l'ordinamento "in loco".

Vantaggi •

In Caso $\Omega(n \log n)$ -

Svantaggi • $O(n^2)$ •

Riunisce i vantaggi del Selection sort (1) (ordinamento in loco) e del Merge sort (2) (ridotto tempo di esecuzione). Ha però lo svantaggio dell'elevato costo computazionale nel caso peggiore.

PROCEDIMENTO •

Quicksort - 2

Ricorsione • Anche l'algoritmo ***quicksort*** è un algoritmo ricorsivo

Divide Et Impera • che adotta una tecnica algoritmica detta ***divide et impera***:

Steps •

① DIVIDE ~ • ***divide***: nella sequenza di n elementi si seleziona un ***pivot***. La sequenza viene quindi divisa in due sottosequenze: quella degli elementi minori o uguali del pivot, e quella degli elementi maggiori o uguali del pivot;

② IMPERA ~ • ***impera***: le due sottosequenze vengono ordinate ricorsivamente;

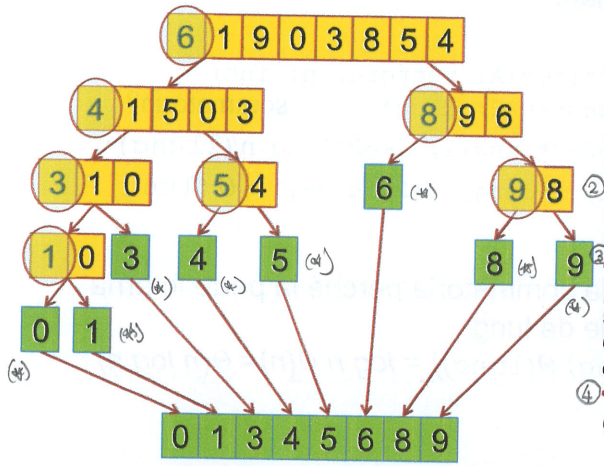
③ PASSO BASE ~ • ***passo base***: la ricorsione procede fino a quando le sottosequenze sono costituite da un solo elemento;

④ COMBINA • ***combina***: non occorre.

- QUANDO TUTTI GLI ELEMENTI RAGGIUNGONO IL PASSO BASE, ESSI SONO GIÀ ORDINATI !!

Graficamente •

Quicksort - 3



① •***divide***: nella sequenza di n elementi seleziona un ***pivot***. La sequenza viene quindi divisa in due sottosequenze: quella degli elementi minori o uguali del pivot, e quella degli elementi maggiori o uguali del pivot;

② •***impera***: le due sotto-sequenze vengono ordinate ricorsivamente;

③ •***passo base***: la ricorsione procede fino a quando le sottosequenze sono costituite da un solo elemento;

④ •***combina***: non occorre

(*) QUANDO RAGGIUNGONO IL PASSO BASE, GLI ELEMENTI SONO GIÀ ORDINATI.

PSEUDOCODICE •

(*) = (LA FUNZIONE Partiziona(...) RITORNA L'INDICE MEDIO IN CORRISPONDENZA DEL QUALE SPEZZARE IL VETTORE, IN QUANTO TUTTI GLI EL. ALLA SUA SX SONO MINORI E QUELLI ALLA DX MAGGIORI DEL PIVOT.

Quicksort - 4

Lo pseudocodice del Quicksort è il seguente:

```
Funzione Quick_sort (A: vettore; ind_primo,
                     ind_ultimo: intero)
  if (ind_primo < ind_ultimo) then
    ind_medio ← Partiziona(A,ind_primo,ind_ultimo)   (*)
    Quick _sort (A,ind_primo,ind_medio)
    Quick _sort (A,ind_medio+1,ind_ultimo)
  return
```

CALCOLO INDICE MEDIO

PASSI RICORSIVI

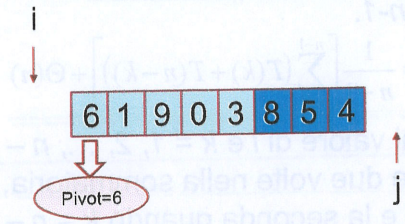
Proprietà Indice Medio •

In questa implementazione ind_medio è ① l'indice dell'estremo superiore della porzione di sinistra (quella contenente elementi minori o uguali del pivot). Il suo valore è ② ***sempre*** compreso fra 1 ed (n-1)

ESEMPIO

Quicksort - 5

Prima di vedere il codice della funzione Partiziona vediamone il funzionamento su un esempio.



FUNZIONE PARTIZIONA (PARTITION)

Quicksort - 6

PSEUDOCODICE

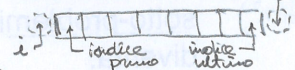
```
Funzione Partiziona (A: vettore; ind_primo, ind_ultimo:
intero)
O(1) pivot ← A[indice_primo]      (A)     //scelta arbitraria
O(1) i ← indice_primo - 1;        (B)
O(1) j ← indice_ultimo + 1
O(n) while true                   (C)
O(1)   repeat
O(1)     j ← j - 1
O(1)   until A[j] ≤ pivot
O(1)   repeat
O(1)     i ← i + 1
O(1)   until A[i] ≥ pivot
O(1)   if (i < j) scambia A[i] e A[j]
O(1) else return j
```

IMPORTANTE! SOSTANZIALMENTE, IL CICLO WHILE TERMINA QUANDO i=j OPPURE SI SONO SCAMBIATI GLI EL.

Note

(A): LA SCELTA DEL PIVOT È ASSOLUTAMENTE ARBITRARIA! PER SEMPLICITÀ, ANDIAMO A CONSIDERARE IL PRIMO ELEMENTO.

(B): POSIZIONIAMO GLI INDICI INIZIALI 1 STEP ESTERNO RISPETTO AGLI ESTREMI DELL'ARRAY.



(C): CI SI CONTINUA A SPOSTARE FINO AL CENTRO DELL'ARRAY DA AMBO LE PARTI E SI SCAMBIANO GLI EL.

COSTO COMPUTAZIONALE

Quicksort - 7

Step by Step

Analizziamo il costo computazionale della funzione `Partiziona()`.

- Le prime tre istruzioni costano $\Theta(1)$.
- Il `while` ha un costo $O(n)$ più un costo pari alla somma dei costi di ciò che avviene al suo interno, che è $\Theta(n)$ poiché:
  - ciascuna iterazione di ognuno dei due `repeat` costa $\Theta(1)$ e avvicina di una posizione un indice all'altro;
  - quindi complessivamente si effettuano $\Theta(n)$ iterazioni dei due `repeat`;
  - terminati i due `repeat` si trova un `if` ed un possibile scambio di elementi, $\Theta(1)$

Costo Finale

- Dunque il costo di `Partiziona()` è $\Theta(n)$.

limiti indice j

- NOTA: il valore j restituito da `Partiziona()` vale:
  - 1 se il pivot è più piccolo degli altri elementi;
  - (n-1) se il pivot è più grande degli altri elementi.

COSTO COMPUTAZIONALE

Quicksort - 8

Valutiamo ora il costo computazionale del Quicksort:

PSEUDOCODICE

```
Funzione Quick_sort (A: vettore;
                     ind_primo, ind_ultimo)
   if (ind_primo < ind_ultimo)                          Θ(1)
      then
         ind_medio ← Partiziona(A,ind_primo,ind_ultimo) Θ(n)
         Quick _sort (A,ind_primo,ind_medio)            T(k)
         Quick _sort (A, ind_medio+1,ind_ultimo)        T(n-k)
   return                                               Θ(1)
```

EQUAZIONE DI RICORRENZA

$T(n)=T(k)+T(n-k)+\Theta(n)$
$T(1)=\Theta(1)$

→ non sappiamo risolverla con alcuno dei metodi studiati…

… USIAMO ALLORA MAGGIORAZIONI e MINORAZIONI …

MAGGIORAZIONI e MINORAZIONI.

Quicksort - 9

Possiamo facilmente derivare la soluzione per due situazioni, il caso migliore e quello peggiore.

Caso Migliore. $\Theta(n \log n)$

- **Caso migliore:**

è quello in cui, ad ogni passo, la dimensione dei due sotto-problemi è identica. L'equazione di ricorrenza diventa:

$T(n) = 2T(n/2) + \Theta(n)$ che ha soluzione $T(n) = \Theta(n \log n)$

Caso Peggiore. $\Theta(n^2)$

- **Caso peggiore:**

è quello in cui, ad ogni passo, la dimensione di uno dei due sotto-problemi da risolvere è 1. L'equazione di ricorrenza diventa:

$T(n) = T(n-1) + \Theta(n)$ che ha soluzione $T(n) = \Theta(n^2)$

CASO MEDIO.

Quick Sort - 10

Considerazioni Ipotesi.

Valutiamo ora il costo computazionale nel caso medio, nell'ipotesi che il valore del pivot suddivida con uguale probabilità $1/(n-1)$ la sequenza da ordinare in due sotto-sequenze di dimensioni k ed $n-k$, per tutti i valori di k tra 1 ed $n-1$.

Equazione di Ricorrenza.

$$T(n) = \frac{1}{n-1}\left[\sum_{k=1}^{n-1}(T(k)+T(n-k))\right]+\Theta(n)$$

Semplificazione.

Ora, per ogni valore di i e $k = 1, 2, \ldots, n-1$ il termine $T(k)$ compare due volte nella sommatoria, la prima quando $k = i$ e la seconda quando $k = n - i$. (*)

Valutiamo dunque il valore di:

$$T(n) = \frac{2}{n-1}\sum_{q=1}^{n-1}T(q)+\Theta(n)$$

(*) $k=1 \Rightarrow T(1)+T(n-1)$
$k=n-1 \Rightarrow T(n-1)+T(1)$
SONO UGUALI! QUINDI VENGONO CONTATI 2 VOLTE.

Metodo di Sostituzione

Quick Sort - 11

Utilizziamo il metodo di sostituzione, e quindi

① Sostituzione Costanti. eliminiamo innanzi tutto la notazione asintotica:

$$T(n) = \frac{2}{n-1}\sum_{q=1}^{n-1}T(q)+\Theta(n) \longrightarrow T(n) = \frac{2}{n-1}\sum_{q=1}^{n-1}T(q)+hn \qquad \vartheta(n) = hn$$

$$T(1) = \Theta(1) \qquad T(1) = k \qquad \vartheta(1) = k$$

② Calcolo di T(2). Per ragioni che saranno chiare tra breve, calcoliamo anche

$$T(2) = \frac{2}{2-1}\sum_{q=1}^{1}T(1) + 2h = \frac{2}{1}k + 2h = 2k + 2h.$$

③ Ipotesi Soluzione. Ipotizziamo ora la soluzione:

$$T(n) \le an \log n$$

Quicksort - 12

④ Sostituzione in Caso Base... ... anzi in T(2)!

Sostituiamo la soluzione innanzi tutto nel caso base.

Visto che $\log 1 = 0$, non possiamo utilizzare $T(1)$ e sostituiamo quindi in $T(2)$, ottenendo:

$$T(2) = 2k + 2h \le 2a \log 2 = 2a$$

che è vera per a opportunamente grande ($a \ge k + h$).

Quicksort - 13

⑤ Sostituzione in Formulazione Asintotica

Per il passo induttivo possiamo scrivere: $T(n) = an\log n$

$$T(n) = \frac{2}{n-1}\sum_{q=1}^{n-1} T(q) + hn \le \frac{2}{n-1}\sum_{q=1}^{n-1}(aq\log q) + hn =$$

$$= \frac{2a}{n-1}\sum_{q=1}^{n-1}(q\log q) + hn$$

Valutiamo ora la sommatoria $\sum_{q=1}^{n-1}(q\log q)$, che spezziamo in due:

$$\sum_{q=1}^{n-1}(q\log q) = \sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1}(q\log q) + \sum_{q=\lceil \frac{n}{2}\rceil}^{n-1}(q\log q)$$

(A) (B)

$\le \log\frac{n}{2} = \log n - 1$ $\qquad$ $\le \log n$

$\log n/2 = \log n - \log 2 = \log n - 1$

NOTE

(A): DATO CHE IL LIMITE SUPERIORE DELLA SOMMATORIA È $n/2$, $\log q$ NON POTRÀ MAI ESSERE $> \log n/2$!

(B) DATO CHE IL LIMITE SUP. È $n-1$ $\log q \le \log n$!

Quicksort - 14

Dunque possiamo scrivere:

SENZA QUESTO TERMINE NEGATIVO, NON SAREMMO IN GRADO DI GIUNGERE ALLA SOLUZIONE.

$$\sum_{q=1}^{n-1}(q\log q) \le \sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1} q(\log n - 1) + \sum_{q=\lceil \frac{n}{2}\rceil}^{n-1} q\log n =$$

$$= (\log n - 1)\sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1} q + \log n \sum_{q=\lceil \frac{n}{2}\rceil}^{n-1} q =$$

$$= \log n \sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1} q - \sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1} q + \log n \sum_{q=\lceil \frac{n}{2}\rceil}^{n-1} q =$$

$\lceil n/2 \rceil$ È IL PIÙ GRANDE INTERO MINORE O UGUALE DI $n/2$!!

$$= \log n \sum_{q=1}^{n-1} q - \sum_{q=1}^{\lceil \frac{n}{2}\rceil - 1} q \le \log n \sum_{q=1}^{n-1} q - \sum_{q=1}^{\frac{n}{2}-1} q$$

(*) (**) (**) (*)

Nota: l'ultima disuguaglianza è vera perché $\lceil \frac{n}{2}\rceil - 1 \ge \frac{n}{2} - 1$

(A) CI SIAMO RICONDOTTI A UNA FORMULA CONTENENTE SOLO SUCCESSIONI DI GAUSS... E CHE POSSIAMO RISOLVERE IN MODO MOLTO SEMPLICE.

Quicksort - 15

Ora,

$$\log n \sum_{q=1}^{n-1} q - \sum_{q=1}^{\frac{n}{2}-1} q = \log n \frac{(n-1)n}{2} - \frac{1}{2}\left(\frac{n}{2} - 1\right)\frac{n}{2} =$$

$$= \frac{1}{2}n(n-1)\log n - \frac{1}{4}\left(\frac{n}{2} - 1\right)n \le$$

$$\le \frac{1}{2}n(n-1)\log n - \frac{1}{4}\left(\frac{n}{2} - 1\right)(n-1) =$$

$$= (n-1)\left(\frac{1}{2}n\log n - \frac{n}{8} + \frac{1}{4}\right)$$

Ricapitolando:

$$\sum_{q=1}^{n-1}(q\log q) \le \log n \sum_{q=1}^{n-1} q - \sum_{q=1}^{\frac{n}{2}-1} q \le (n-1)\left(\frac{1}{2}n\log n - \frac{n}{8} + \frac{1}{4}\right)$$

(*): AVENDO USATO LE MAGGIORAZIONI E IL METODO DI SOSTITUZIONE, POSSIAMO SOLO RICAVARE UN LIMITE SUPERIORE: QUINDI UN $O(\ldots)$.

Quicksort - 16

Sapendo quindi che:

$$T(n) \le \frac{2a}{n-1}\sum_{q=1}^{n-1}(q\log q) + hn$$

e che:

$$\sum_{q=1}^{n-1}(q\log q) \le (n-1)\left(\frac{1}{2}n\log n - \frac{n}{8} + \frac{1}{4}\right)$$

possiamo scrivere:

$$T(n) \le \frac{2a}{n-1}(n-1)\left(\frac{1}{2}n\log n - \frac{n}{8} + \frac{1}{4}\right) + hn = an\log n - \frac{an}{4} + \frac{a}{2} + hn$$

Scegliendo a sufficientemente grande si ha che $hn - \frac{an}{4} + \frac{a}{2} \le 0$ e per tale a si avrà:

$$T(n) \le an\log n - \frac{an}{4} + \frac{a}{2} + hn \le an\log n$$

Il che ci permette di dimostrare che $\boldsymbol{T(n) = O(n\log n)}$. (*)
Poiché già sappiamo che $T(n) = \Omega(n\log n)$ perché il costo del caso migliore limita superiormente il costo computazional degli altri casi, abbiamo che nel caso medio il Quicksort ha un costo computazionale:

$$\boldsymbol{T(n) = \Theta(n\log n)}$$

Quicksort - 17

EQUIPROBABILITÀ del PIVOT

Osservazioni

Dati Input Disordinati → $\Theta(n \log n)$

- L'analisi ora fatta è valida nell'ipotesi che il valore del pivot sia equiprobabile e, quando questo è il caso, **il quicksort è considerato l'algoritmo ideale per input di grandi dimensioni**.

Dati Input Semiordinati → $\Theta(n^2)$

- A volte però l'ipotesi di equiprobabilità non è soddisfatta (ad esempio quando i valori in input sono "poco disordinati") e le prestazioni dell'algoritmo degradano.

...

QUASI CONTRO LOGICA, IL QUICKSORT È UN ALGORITMO CHE NON TRAE VANTAGGIO MA ANZI... TRAE SVANTAGGIO NEL LAVORARE SU UNA SEQUENZA PARZIALMENTE ORDINATA!

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Esercizi per casa

SAPIENZA
UNIVERSITÀ DI ROMA

RENDERE L'ALGORITMO INDIPENDENTE (*): DARE INPUT NON SIGNIFICA CHE NON CADREMO PIÙ NEL CASO PESSIMO... SEMPLICEMENTE, IL CASO PESSIMO DIVENTA UN CASO CHE HA LA STESSA PROBABILITÀ DI TUTTI GLI ALTRI CASI!

Quicksort - 18

...

Per ovviare a tale inconveniente si possono adottare delle tecniche volte a ***randomizzare*** la sequenza da ordinare, cioè volte a disgregarne l'eventuale regolarità interna.

Tali tecniche mirano a rendere l'algoritmo indipendente (*) dall'input, e quindi consentono di ricadere nel caso medio.

Alcune di tali tecniche sono:

- prima di avviare l'algoritmo, alla sequenza da ordinare viene applicata una permutazione degli elementi generata casualmente;
- l'operazione di partizionamento sceglie casualmente come pivot il valore di uno qualunque degli elementi della sequenza anziché sistematicamente il valore di quello più a sinistra.

Esercizi - 1

✓ ① • Sia dato un vettore di lunghezza *n* contenente solo valori 0 e 2. Si progetti un algoritmo con costo computazionale lineare che modifichi il vettore in modo che tutte le occorrenze di 0 si trovino più a sinistra di tutte le occorrenze di 2.

✓ ② • Si considerino i valori 0 1 2 3 4 5 6 7. Si determini una permutazione di questi valori che generi il caso peggiore per l'algortimo QuickSort.

✓ ③ • Calcolare il costo computazionale del QuickSort nel caso in cui il vettore contenga tutti elementi uguali ed in cui sia già ordinato da destra a sinistra.

Esercizi - 2

④ • Si progetti un algoritmo il più efficiente possibile (*) per il seguente problema:
  - Data una matrice *m x n*, si vogliono rimescolare i suoi elementi in modo che tutti i vettori riga e tutti i vettori colonna siano ordinati in senso non decrescente.

△ Vedi Risoluzioni Esercizi in Codici PyTHON e SU APPUNTI △

(*) COME DA TEORIA, LA MASSIMA EFFICIENZA POSSIBILE PER UN ALGORITMO DI ORDINAMENTO BASATO SUI CONFRONTI HA COSTO COMPUTAZIONALE $O(m \log m)$.
- DOVREMO QUINDI USARE IL QuickSort O IL MergeSort

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Il problema dell'ordinamento: L'Heapsort

Tiziana Calamoneri

SAPIENZA
UNIVERSITÀ DI ROMA

Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

SOLUZIONE ES.4

Esercizio svolto- 1

Data una matrice *m x n*, si vogliono rimescolare i suoi elementi in modo che tutti i vettori riga e tutti i vettori colonna siano ordinati in senso non decrescente.

Soluzione 1. Consideriamo la matrice come un unico lungo vettore e ordiniamolo con un algoritmo efficiente:

21	13	2	8
12	7	5	17
3	9	19	14

21	13	2	8	12	7	5	17	3	9	19	14	
2	3	5	7	8	9	12	13	14	17	19	21	

2	3	5	7
8	9	12	13
14	17	19	21

$T(n)=\Theta(nm \log nm)=\Theta(nm \log n+nm \log m)$

Esercizio svolto- 2

STEPS RICORSIVI

Soluzione 2. Ordino ogni riga ed ogni colonna separatamente con un algoritmo efficiente:

21	13	2	8
12	7	5	17
3	9	19	14

① →

2	8	13	21
5	7	12	17
3	9	14	19

② →

2	7	12	17
3	8	13	19
5	9	14	21

COSTO COMPUTAZ.

$T(n)=\Theta(m\ n \log n+ n\ m \log m)=\Theta(nm \log n+nm \log m)$

①: ORDINA LE RIGHE
②: ORDINA LE COLONNE

HEAPSORT

PROPRIETÀ

Heapsort - 1

VANTAGGI

L'algoritmo ***heapsort*** è un algoritmo di ordinamento piuttosto complesso che esibisce ottime caratteristiche:

$O(n \log n)$ ~ • come Mergesort ha un costo computazionale di $O(n \log n)$ anche nel caso peggiore

IN LOCO ~ • come Selection sort ordina in loco. (COME ANCHE IL QUICKSORT!)

REQUISITI

STRUTTURE DATI HEAP ~ Sfrutta una opportuna organizzazione dei dati, ossia una ***struttura dati***, che garantisce una o più specifiche proprietà, il cui mantenimento è *essenziale* per il corretto funzionamento dell'algoritmo.

↓

Struttura dati Heap

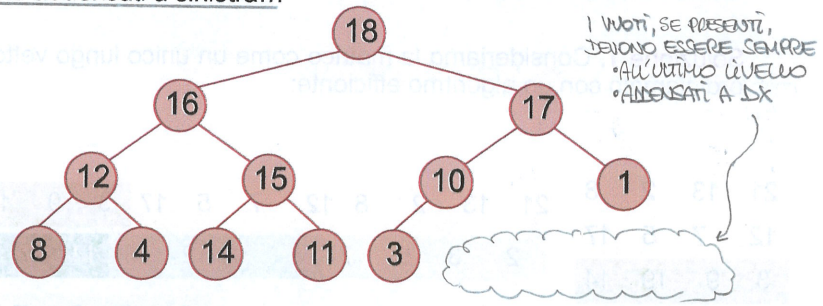
STRUTTURA DATI HEAP

DEFINIZIONE.

La struttura dati Heap - 1

Albero Binario, Completo o Quasi Completo

Uno ***heap*** è un albero binario completo o quasi completo, ossia un albero binario in cui tutti i livelli sono pieni, tranne l'ultimo, i cui nodi sono addensati a sinistra...



I NODI, SE PRESENTI, DEVONO ESSERE SEMPRE
- ALL'ULTIMO LIVELLO
- ADDENSATI A DX

Ordinamento Verticale

Con l'ulteriore proprietà che la chiave di ogni nodo è ***maggiore o uguale*** alla chiave dei suoi figli (proprietà di ordinamento verticale).

VETTORE di MEMORIZZAZIONE

La struttura dati Heap - 2

Proprietà Numero Indici

Il modo più naturale per memorizzare uno heap è utilizzare un vettore *A*, con indici che vanno da *1* fino al numero di nodi dell'heap, ***heap_size***, i cui elementi possono essere messi in corrispondenza con i nodi dell'heap:

Ordine di Riempimento

- il vettore è riempito a partire da sinistra; se contiene più elementi del numero *heap_size* di nodi dell'albero, allora i suoi elementi di indice > *heap_size* non fanno parte dell'heap;
- (continua)

La struttura dati Heap - 3

- (continua)

Univocità Nodo-Elem.
- ogni nodo dell'albero binario corrisponde a uno e un solo elemento del vettore *A*;

Radice Albero
- la radice dell'albero corrisponde ad *A[1]*; (A[0])

Figlio Sx
- il figlio sinistro del nodo che corrisponde all'elemento *A[i]*, se esiste, corrisponde all'elemento A[2i]: ***left(i) = 2i***; (2i+1)

Figlio Dx
- il figlio destro del nodo che corrisponde all'elemento *A[i]*, se esiste, corrisponde all'elemento *A[2i + 1]*: ***right(i) = 2i+1***; (2i+2)

Padre
- il padre del nodo che corrisponde all'elemento A[i] corrisponde all'elemento A[$\lfloor i/2 \rfloor$]: ***parent(i)*** = $\lfloor i/2 \rfloor$. (i/2-1)

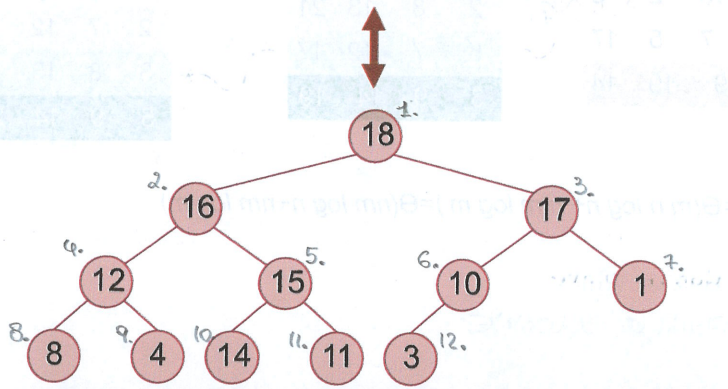
Graficamente

La struttura dati Heap - 4

Vettore HEAP

1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.
18	16	17	12	15	10	1	8	4	14	11	3

Albero HEAP



Ordine Elementi

Da SX a DX

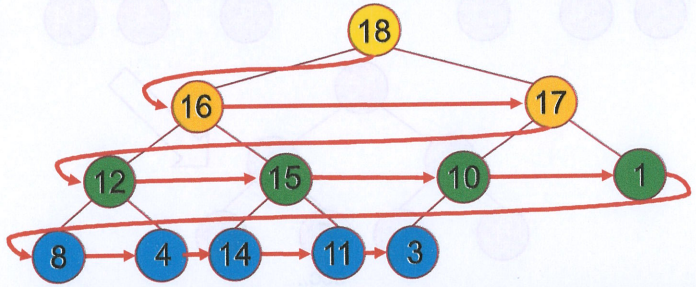
La struttura dati Heap - 5

Si noti che scorrere il vettore da sinistra a destra corrisponde a muoversi sull'albero per livelli, dall'alto verso il basso e da sinistra a destra in ciascun livello

Vettore HEAP

1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.
18	16	17	12	15	10	1	8	4	14	11	3

Albero HEAP



Da DX a SX

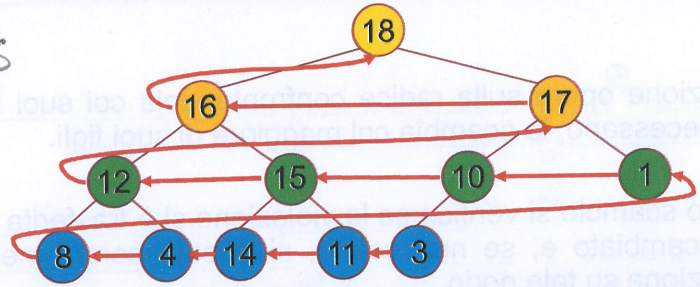
La struttura dati Heap - 6

Simmetricamente, scorrere il vettore da destra a sinistra corrisponde a muoversi sull'albero per livelli, dal basso verso l'alto e da destra a sinistra in ciascun livello

Vettore HEAP

1.	2.	3.	4.	5.	6.	7.	8.	9.	10.	11.	12.
18	16	17	12	15	10	1	8	4	14	11	3

Albero HEAP



La struttura dati Heap - 7

PROPRIETÀ

Proprietà:

Altezza $\Theta(\log n)$

- Poiché lo heap ha tutti i livelli completamente pieni tranne al più l'ultimo, la sua altezza è $\Theta(\log n)$.

Ordinamento Verticale

- Con questa implementazione, la proprietà di ordinamento verticale implica che per tutti gli elementi tranne *A[1]* (poiché esso corrisponde alla radice dell'albero e quindi non ha genitore) vale:

$$A[i] \leq A[parent(i)].$$

El. Max nella Radice

- L'elemento massimo risiede nella radice, quindi può essere trovato in tempo $O(1)$.
  - SI ACCEDE SEMPRE DIRETTAMENTE ALL'ELEMENTO AVENTE INDICE $\emptyset$ (A[$\emptyset$])

Funzione ausiliarie

FUNZIONI AUSILIARIE

Necessarie x Funzionamento

L'algoritmo Heapsort si avvale di due funzioni ausiliarie, necessarie per il suo corretto funzionamento

HEAPIFY

- Funzione Heapify

BUILDHEAP

- Funzione Buildheap

Illustreremo tali funzioni prima di descrivere l'algoritmo Heapsort.

FUNZIONE HEAPIFY

Funzione Heapify - 1

SCOPO • La funzione Heapify ha lo scopo di mantenere la proprietà di heap, sotto l'ipotesi che nell'albero su cui viene fatta lavorare sia garantita la proprietà di heap per entrambi i sottoalberi (sinistro e destro) della radice. Di conseguenza l'unico nodo che può violare la proprietà di heap è la radice dell'albero, che può essere minore di uno o di entrambi i figli.

IPOTESI BASE •

STEPS •

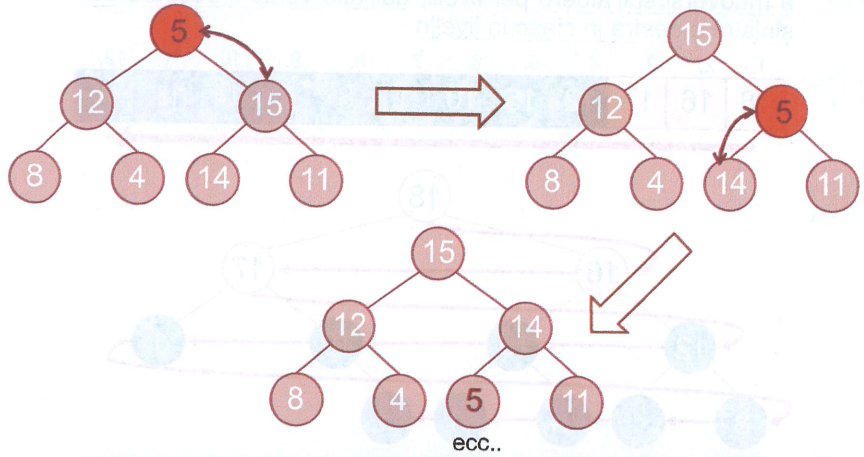
La funzione ① opera sulla radice confrontandola coi suoi figli e, se necessario, ② la scambia col maggiore di suoi figli.

Dopo lo scambio ③ si verifica se la violazione si è trasferita sul figlio scambiato e, se necessario, si ripete ricorsivamente ④ l'operazione su tale nodo.

GRAFICAMENTE •

Funzione Heapify - 2

Esempio:



ecc..

PSEUDOCODICE •

Funzione Heapify - 3

heapsize

Ⓐ: SI ESTRAGGONO I FIGLI DX e SX DELL'ELEMENTO A[i].
Ⓑ: SI MEMORIZZA L'INDICE DEL MASSIMO TRA PADRE E FIGLIO SX
Ⓒ: SI MEMORIZZA L'INDICE DEL MASSIMO INCLUDENDO IL FIGLIO DX
Ⓓ: SE INDICE MAX ≠ INDICE PADRE, SI SCAMBIANO I DUE

```
Funzione Heapify (A: vettore; i: intero)
   L ← left(i); R ← right(i)                          Ⓐ
   if ((L ≤ heap_size) and (A[L] > A[i]))             Ⓑ
      indice_max ← L
   else
      indice_max ← i
   if ((R ≤ heap_size) and (A[R] > A[indice_max]))    Ⓒ
      indice_max ← R
   if (indice_max ≠ i)                                Ⓓ
      scambia A[i] e A[indice_max]
      Heapify (A, indice_max)                         Ⓔ
   return
```

Θ(1)

T(n')

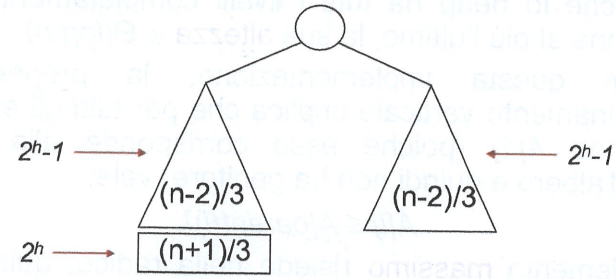
heapsize

COSTO COMPUTAZIONALE

Equazione di Ricorrenza ~ T(n) = Θ(1) + T(n'), dove n' è il numero di nodi del sottoalbero che ha più nodi.

Funzione Heapify - 4

Numero Max di Nodi ~ I sottoalberi della radice non possono avere più di *2n/3* nodi, situazione che accade quando l'ultimo livello è pieno esattamente a metà:



O(log n) ~ Quindi l'equazione di ricorrenza diventa $T(n)=T(2/3\ n)+\Theta(1)$, (*) *che ha soluzione:*

$$T(n)=O(\log n)$$

(*): SI RISOLVE USANDO IL METODO PRINCIPALE, BASATO SUL TEOREMA MASTER.

Funzione Buildheap - 1

FUNZIONE BUILDHEAP

SCOPO

La funzione Buildheap serve per ① trasformare qualunque vettore contenente n elementi in uno heap, ② chiamando ripetutamente Heapify sugli opportuni nodi dello heap.

PROPRIETÀ PRINCIPALI

Osservazioni:

- poiché Heapify presuppone che entrambi i sottoalberi della radice siano heap, deve essere chiamata scorrendo l'albero per livelli dal basso verso l'alto (quindi, sul vettore, da destra a sinistra);
- ogni foglia è già uno heap, quindi basta chiamare Heapify a partire dal nodo interno più a destra, che ha indice:

$$\lfloor n/2 \rfloor$$

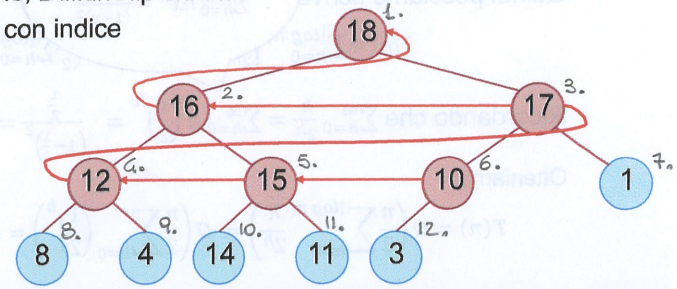
Funzione Buildheap - 2

PSEUDOCODICE

```
Funzione Build_heap (A: vettore)
   for i = ⌊n/2⌋ downto 1
      Heapify (A, i)
   return
```

ESEMPIO

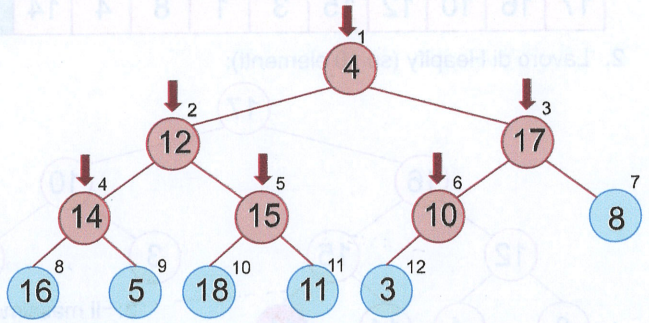
In questo esempio, Buildheap chiama Heapify sui nodi con indice 6, 5, 4, 3, 2, 1:



Funzione Buildheap - 3

```
Funzione Build_heap (A: vettore)
   for i = ⌊n/2⌋ downto 1
      Heapify (A, i)
   return
```

i=6



Funzione Buildheap - 4

```
Funzione Build_heap (A: vettore)
   for i = ⌊n/2⌋ downto 1
      Heapify (A, i)
   return
```

COSTO COMPUTAZIONALE

La funzione Build_heap effettua $\Theta(n)$ chiamate di Heapify, che sappiamo avere ciascuna costo $O(\log n)$, quindi:

Calcolo Approssimato ~ $O(n \log n)$

$$T(n) = O(n \log n)$$

Calcolo Accurato $O(n)$ ~ Con un calcolo più accurato si può mostrare che $T(n)=\Theta(n)$

Funzione Buildheap - 5

Dimostrazione~

Mostriamo che $T(n)=O(n)$.

- Il tempo richiesto da Heapify applicata ad un nodo che è radice di un albero di altezza h è $O(h)$ per quanto già detto;
- il numero di nodi che sono radici di un sottoalbero di altezza h è al massimo $\left\lceil \frac{n}{2^{h+1}} \right\rceil$.

Se $|x| < 1$: $\sum_{h=0}^{\infty} hx^h = \frac{x}{(1-x)^2}$

Quindi possiamo scrive

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \lceil \frac{n}{2^h} \rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h}\right)$$

Ricordando che $\sum_{h=0}^{\infty} \frac{h}{2^h} = \sum_{h=0}^{\infty} h\left(\frac{1}{2}\right)^h = \frac{\frac{1}{2}}{\left(1-\frac{1}{2}\right)^2} = 2$

Otteniamo:

$$T(n) = O\left(\frac{n}{2} \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h}\right) = O\left(\frac{n}{2} \sum_{h=0}^{\infty} \left(\frac{1}{2}\right)^h\right) = O\left(\frac{n}{2} 2\right) = O(n)$$

Funzione Heapsort - 1

FUNZIONE HEAPSORT •

Vediamo ora il funzionamento di Heapsort.

STEPS •

① CREAZIONE HEAP • Trasforma un vettore A di dimensione n in un heap (di n nodi), mediante Build_heap.

② SCAMBIO A[1] CON A[n] • Ora il max del vettore è in $A[1]$ e, per metterlo nella corretta posizione dell'ordinamento, basta scambiarlo con $A[n]$. (COSTO COMPUTAZ. $O(1)$)

③ RIDUCI A (n-1) ELEMENTI • La dim. dell'heap viene ridotta ad $(n-1)$, e:
- i due sottoalberi della radice sono ancora degli heap
- solo la nuova radice (ex foglia più a destra) può violare la proprietà del nuovo heap di dimensione (n – 1).

④ TRASFORMA IN HEAP GLI (N-1) ELEMENTI • Ripristina la proprietà di heap sui residui $(n-1)$ elementi con Heapify;

⑤ SCAMBIO A[1] CON A[n-1] • Scambia il nuovo max $A[1]$ col penultimo elemento;

⑥ ...E COSÌ VIA ... • Riapplica il procedimento riducendo via via la dimensione dell'heap a $(n-2)$, $(n-3)$, ecc., fino ad arrivare a 2.

Esempio di Heapsort - 1

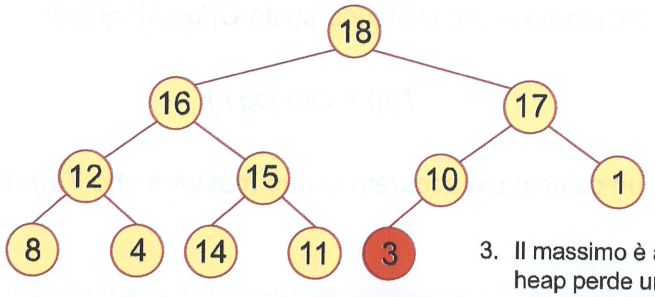
ESEMPIO • 1° Iterazione

Prima iterazione, Heap_size= 12

1. Scambio del massimo:

18	16	17	12	15	10	1	8	4	14	11	3

2. Lavoro di Heapify (su 11 elementi):



3. Il massimo è a posto, lo heap perde un elemento

Esempio di Heapsort - 2

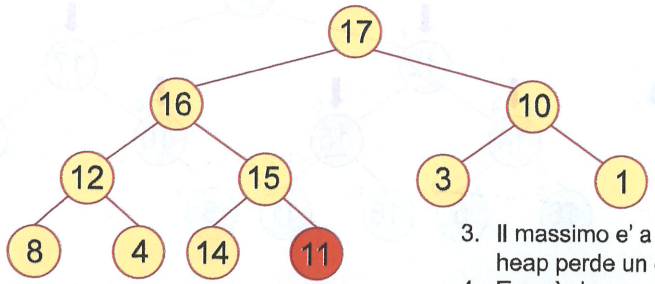
2° Iterazione

Seconda iterazione, Heap_size= 11

1. Scambio del massimo:

17	16	10	12	15	3	1	8	4	14	11	18

2. Lavoro di Heapify (su 10 elementi):



3. Il massimo e' a posto, lo heap perde un elemento
4. E così via...

E così via...

Heapsort - 9

PSEUDOCODICE • Vediamo ora lo pseudocodice di Heapsort:

```
Funzione Heapsort (A: vettore)
  Build_heap(A)}Ⓐ                          O(n)
  for heap_size = n downto 2               (n-1) iterazioni + Θ(1)
    scambia A[1] e A[heap_size]}Ⓑ          Θ(1)
    Heapify(A, heapsize-1, 1)}Ⓒ            O(log n)
  return                                   Θ(1)
```

COSTO COMPUTAZIONALE O(n log n) •

$T(n) = O(n) + (n-1)(\Theta(1) + O(\log n)) + \Theta(1) + \Theta(1) = O(n \log n)$

ESERCIZI •

Esercizi

① • Progettare un algoritmo che, dato in input un vettore che rappresenta un heap, restituisca il valore minimo. Fare le opportune considerazioni sul costo computazionale

② • Un Heap minimo è un albero binario completo o quasi completo con la proprietà che la chiave su ogni nodo è minore o uguale alla chiave dei suoi figli. Si modifichi l'algoritmo di Heap sort in modo che la struttura dati di riferimento sia un heap minimo e non un heap.

▶ Vedi Risoluzioni Esercizi in Codici PYTHON e SU APPUNTI ◀

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Esercizi per casa

SAPIENZA
UNIVERSITÀ DI ROMA

COMMENTI PseudoCodice

Ⓐ: TRASFORMARE IL VETTORE IN INPUT A IN UNA STRUTTURA DATI DI TIPO HEAP
- L'ALGORITMO HEAPSORT PUÒ FUNZIONARE SOLO CON STRUTTURE DATI DI TIPO HEAP.

Ⓑ: SI SCAMBIA IL PRIMO ELEMENTO DEL SOTTO-VETTORE CON L'ULTIMO ELEMENTO
- IN QUESTO MODO SI SPOSTANO TUTTI GLI ELEMENTI DI VALORE MAGGIORE VERSO LA DESTRA DELL'ARRAY.

Ⓒ: SI RIDUCE IL VETTORE DI UN ELEMENTO E LO SI TRASFORMA IN UN HEAP.

* Vedi Algoritmo HeapSort in Python nel File "HeapSort.py" *

Corso di laurea in Informatica
Introduzione agli Algoritmi
Lezioni in modalità mista o a distanza

Il problema dell'ordinamento: Ordinamenti lineari

Tiziana Calamoneri

SAPIENZA
UNIVERSITÀ DI ROMA

Slides realizzate sulla base di quelle preparate da T. Calamoneri e G. Bongiovanni per il corso di Informatica Generale tenuto a distanza nell'A.A. 2019/20

ALGORITMI LINEARI

Algoritmi lineari

ORDINAMENTO PER CONFRONTI $\Omega(n \log n)$

- Abbiamo dimostrato il teorema che asserisce che ogni algoritmo di ordinamento ***che opera per confronti*** ha un costo computazionale di $\Omega(n \log n)$.

ORDINAMENTO LINEARE $O(n)$

- Com'è possibile, allora, avere degli algoritmi di ordinamento di costo computazionale lineare?

BISOGNA EVITARE I CONFRONTI!

- Si possono fare ipotesi aggiuntive che ci permettano di evitare che l'algoritmo sia basato sui confronti.

ESEMPI PRINCIPALI
- COUNTING SORT
- BUCKET SORT

COUNTING SORT ~ Versione Classica

PROPRIETÀ

Counting Sort - 1

IPOTESI M VALORI [1...K]

- **Ipotesi**: ciascuno degli n elementi da ordinare è un intero di valore compreso in un intervallo $[1..k]$ (*)

COSTO COMPUTAZ. $K \le n \Rightarrow \Theta(n+K)$ $K \le n \Rightarrow \Theta(n)$

- Il costo computazionale è di $\Theta(n+k)$. Se $k = O(n)$ allora l'algoritmo ordina n elementi in tempo lineare, cioè $\Theta(n)$.

IDEA di BASE: USO VALORI PER DETERMINARE INDICE POSIZIONE

- **Idea**: fare in modo che il valore di ogni elemento della sequenza determini direttamente la sua posizione nella sequenza ordinata.

NOTE AGGIUNTIVE

(*): SE ANCHE I VALORI NON FOSSERO INTERI, LI SI POSSONO MOMENTANEAMENTE RENDERE INTERI (E.G.: MOLTIPLICANDOLI PER 10/1000/...) E, CONCLUSO L'ALGORITMO, RITRASFORMARLI IN NUMERI REALI (E.G. DIVIDENDOLI PER 10/1000/...)

ESEMPIO

Counting Sort - Esempio

A:	8	6	7	2	5	6	1	8	4	4	1	6

n=12, k=8

C:	0	0	0	0	0	0	0	0

1) Scorrendo A conteggiamo in C il numero di occorrenze di ciascun valore

Nota: $\sum_{i=1}^{k} C[i] = n$

	1	2	3	4	5	6	7	8
C:	2	1	0	2	1	3	1	2

2) Scorrendo C ricopiamo in A ciascun indice di C tante volte quanto è il valore in C di quell'indice

A:	1	1	2	4	4	5	6	6	6	7	8	8

Counting Sort - 2

PSEUDOCODICE

```
Funzione Counting_sort (A: vettore; k, n: intero)
Ⓐ for i = 1 to k                          kΘ(1) + Θ(1)
    C[i] ← 0                              Θ(1)
Ⓑ for j = 1 to n                          nΘ(1) + Θ(1)
    C[A[j]] ← C[A[j]] + 1                 Θ(1)
  //C[i] ora contiene il numero di elementi uguali a i
Ⓒ j = 1                                   Θ(1)
Ⓓ for i = 1 to k                          kΘ(1) + Θ(1)
  Ⓔ while (C[i] > 0)                      (∑_{i=1}^{k} C[i]) Θ(1) + Θ(1)
      A[j] ← i                            Θ(1)
      j ← j + 1                           Θ(1)
      C[i] ← C[i] - 1                     Θ(1)
  return
```

... Vedi Commenti in Slide Successiva ...

COSTO COMPUTAZIONALE

Poiché $\sum_{i=1}^{k} C[i] = n$, $T(n) = \Theta(k) + \Theta(n) + \Theta(k) + \Theta(n) = \Theta(k+n)$

$\Theta(k+n)$.

Counting Sort - 3

LIMITI ALGORITMO

VALIDITÀ SOLO SENZA Dati SATELLITE

- Lo pseudocodice appena illustrato è adeguato solamente se non vi sono dati satellite. Infatti, il ciclo che riscrive il vettore *A* di fatto ne sovrascrive i dati.

CON DATI SATELLITE...

Nuovo Vettore Ausiliario B

- Se ci dovessero essere dati satellite, oltre ai vettori *A* e *C* si deve introdurre un nuovo vettore *B* di *n* elementi, che alla fine conterrà la sequenza ordinata, e si deve usare uno schema leggermente più complesso...

I DATI SATELLITE SONO I METADATA

COMMENTI PSEUDOCODICE

Ⓐ: INIZIALIZZAZIONE VETTORE DI SUPPORTO C CON NUMERO ELEMENTI K (NON N!)
Ⓑ: SI SCORRONO I VALORI DI A E SI CONTEGGIA IL NUMERO DELLE LORO ISTANZE NEGLI INDICI CORRISPONDENTI.
Ⓒ: INIZIALIZZAZIONE INDICE INSERIMENTO VALORI ORDINATI NEL VETTORE A.
Ⓓ: SI SCORRONO I VALORI IN C E SI INSERISCONO IN A UN NUMERO DI OCCORRENZE di i
Ⓔ (Indice) PARI AL VALORE CORRISPONDENTE CONTENUTO IN C

COUNTING SORT - Versione Avanzata

Counting Sort - 4

PROCEDIMENTO

① CONTEGGIO ISTANZE INDICI

② CALCOLO ULTIMI INDICI ORDINATI PER CIASCUN VALORE

- Dopo aver conteggiato gli elementi in *C*, si fa una seconda passata su *C*, da sinistra a destra a partire da *C[2]*, nella quale a ogni elemento *C[i]* si somma il precedente;
- alla fine di tale fase:
  - *C[i]* indica la posizione corretta, nel vettore ordinato, per l'elemento di valore pari a *i* più a destra fra quelli contenuti nel vettore *A*;

Posizione Ultima Istanza Valore

  - *C[i] - C[i-1]* indica quanti elementi ci sono con valore pari a *i*

Numero Istanze Valore i

③ COPIA ELEMENTI A in VETTORE B

si scorre quindi il vettore *A* da destra a sinistra e:

- si copia in *B* ogni elemento *A[j]=k* nella posizione giusta che è *C[k]*; (*)
- si decrementa di 1 il valore *C[k]*.

(*): COPIANDO, ANZICHÉ SOVRASCRIVENDO, NON SI PERDONO I DATI SATELLITE!

Graficamente

Counting Sort – Esempio 2

A:	8	6	7	2	5	6	1	8	4	4	1	6

n=12, k=8 C:

	0	0	0	0	0	0	0	0

① 1) Prima passata: come nell'algoritmo precedente

	1	2	3	4	5	6	7	8
C:	2	1	0	2	1	3	1	2

② 2) Seconda passata: partendo da C[2], ad ogni elemento si somma il precedente

Ci sono due elementi pari a 8 che vanno nelle posizioni 12 e 11

	1	2	3	4	5	6	7	8
C:	2	3	3	5	6	9	10	12

Counting Sort – Esempio 2

③ SI SCORRE A DA DX VERSO SX, SI COPIA IN B OGNI ELEMENTO A[j]=K IN POSIZIONE C[K] E SI DECREMENTA DI 1 IL VALORE C[K]

3.1 A:

1	2	3	4	5	6	7	8	9	10	11	12
8	6	7	2	5	6	1	8	4	4	1	6

A[j]=K
A[12]=6

3.2 C:

1	2	3	4	5	6	7	8
1	3	3	3	6	8	10	11

m=C[K]
m=C[6]=9
C[K]=C[K]-1=8

3.3 B:

1	2	3	4	5	6	7	8	9	10	11	12
	1		4	4				6			8

B[m]=A[j]
B[9]=6

PSEUDOCODICE •

Counting Sort - 4

```
Funzione Counting_sort_con_Dati_Satellite (A, B; k, n)
(A) for i = 1 to k
      C[i] ← 0
(B) for j = 1 to n
      C[A[j]] ← C[A[j]] + 1
    //C[i] ora contiene il numero di elementi uguali a i
(C) for i = 2 to k
      C[i] ← C[i] + C[i-1]
    //C[i] ora contiene il numero di elementi minori o uguali a i
(D) for j = n downto 1
      B[C[A[j]]] ← A[j]
      C[A[j]] ← C[A[j]] - 1
(E) for i=1 to n do
      A[i] ← B[i]
    return
```

(A) ~ INIZIALIZZAZIONE VETTORE C (con K elementi)
(B) ~ CONTEGGIO NUMERO ISTANZE VALORI UGUALI AD i
(C) ~ CONTEGGIO NUMERO ELEMENTI/VALORI ≤ i
(D) ~ COPIA ELEMENTI A[j]=K in B ALLA POSIZIONE C[K]
(E) ~ SOSTITUZIONE B IN A ELEMENTO PER ELEMENTO

COSTO COMPUTAZIONALE •

Il cui costo è chiaramente $T(n) = \Theta(k) + \Theta(n)$

$\Theta(k+n)$ ~

BUCKET SORT

PROPRIETÀ •

Bucket sort - 1

IPOTESI • **Ipotesi**: gli *n* elementi da ordinare sono distribuiti in modo uniforme nell'intervallo *[1..k]*, senza alcuna ipotesi su *k*. (*)

n valori [1..k] • Distribuzione UNIFORME • No limiti su K •

COSTO COMPUTAZ. • Il costo computazionale **medio** è di $\Theta(n)$.

$\Theta(n)$

IDEA di BASE • **Idea**: dividere l'intervallo *[1..k]* in *n* sottointervalli di uguali dimensioni *k/n*, detti ***bucket***, e distribuire i valori nei bucket.

Distribuzione valori in BUCKET •

UNIFORMITÀ • Numero Elementi nei Bucket. Poiché gli elementi in input sono uniformemente distribuiti, non ci si aspetta che molti elementi cadano nello stesso bucket.

NOTE AGGIUNTIVE

(*): K PUÒ AVER QUALSIASI VALORE... ANCHE K>n!! NEL COUNTING SORT, INVECE, È MEGLIO AVERE K≤n

PROCEDIMENTO •

Bucket sort - Esempio

n=8, k=16

A:

1	2	3	4	5	6	7	8
8	6	15	2	5	16	4	9

B:

1-2	3-4	5-6	7-8	9-10	11-12	13-14	15-16
2	4	6 5	8	9			15 16

1) Inserimento degli elementi nei bucket
2) Ordinamento di ogni bucket
3) Copiatura dei bucket ordinati

1	2	3	4	5	6	7	8
2	4	5	6	8	9	15	16

Bucket sort - 2

PSEUDOCODICE.

```
Funzione Bucket_Sort (A: vettore, n: intero)
for i=1 to n                                   nΘ(1)
  inserisci A[i] nella lista B[ ]
for i=1 to n                                   Σ_{i=1}^{n}(costi di ordinamento)
      ordina la lista B[i]
      con insertion_sort
concatena le liste B[1] B[2] … B[n]            Θ(n)
      in quest'ordine
```

(*) Vedi COMMENTI A PSEUDOCODICE IN ULTIMA SLIDE.

COSTO COMPUTAZIONALE

$\vartheta(n)$.

- I costi di ordinamento dipendono dalla lunghezza delle liste.
- Ci sono *n* bucket ed *n* valori; per l'ipotesi di equiprobabilità, in media ci sarà un numero costante di valori in ogni bucket, quindi la lista B[i] ha in media lunghezza costante.
- Quindi, in media T(n) = Θ(n).

ESERCIZI.

Esercizi

① Mostrare che il Counting sort è un algoritmo di ordinamento stabile.

② Qual è il tempo di esecuzione del Bucket sort nel caso peggiore? quale semplice modifica dell'algoritmo consente di conservare tempo medio lineare e costo Θ(n log n) nel caso peggiore?

③ Il bucket sort può essere modificato in modo che l'ordinamento all'interno delle liste sia eseguito tramite counting sort. Affinché il costo dell'algoritmo sia lineare anche nel caso peggiore, quale ipotesi bisogna fare su k?

→ Vedi Risoluzioni Esercizi nel Codice "Esercizi-Alg lineari.py" e su APPUNTI CARTACEI

Corso di laurea in Matematica
Insegnamento di Informatica generale
Lezioni a distanza

Esercizi per casa

SAPIENZA
UNIVERSITÀ DI ROMA

COMMENTI A PSEUDOCODICE

⓪: INIZIALIZZAZIONE VETTORE B CON NUMERO DI ELEMENTI K/n
Ⓐ: INSERIMENTO VALORI A[i] IN CORRISPONENTI BUCKET DI B
Ⓑ: ORDINAMENTO ELEMENTI ALL'INTERNO DI CIASCUN BUCKET TRAMITE UN ALTRO ALGORITMO DI ORDINAMENTO (e.g.: SELECTION SORT)
Ⓒ: CONCATENAZIONE ELEMENTI DEI BUCKETS IN UNA NUOVA LISTA

FILE/DOCS AGGIUNTIVI

Vedi Algoritmi COUNTING SORT e BUCKET SORT nei seguenti Files Python
- CountingSort.py
- BucketSort.py